# NLP Model Deployment Report (FastAPI - Docker)

## 1. Objective

To deploy a pre-trained Natural Language Processing (NLP) model (simulated as a scikit-learn classifier for text sentiment) as a high-performance REST API using the FastAPI framework, and to containerize the application using **Docker**.

## 2. Deployment Process Steps

This process assumes the user has created the project structure and installed the necessary system tools (Python, pip, Docker).

### Step 1: Project Setup and Dependencies

The project requires the following files to be placed in the project root directory: setup_files.py, app.py, requirements.txt, and **Dockerfile**.

1. **Install Libraries (for local testing):** Install all necessary Python libraries defined in requirements.txt.
   pip install -r requirements.txt

2. **Create Model Assets:** Run the setup script (setup_files.py) to simulate and save the trained model (model.pkl) and tokenizer/vectorizer (tokenizer.pkl). **These files must exist before building the Docker image.**
   python setup_files.py

### Step 2: API Creation (app.py)

The app.py script was written using FastAPI, incorporating the following critical components:

- **Pydantic Schemas:** Used TextRequest and PredictionResponse for strict data validation and clear input/output structure. This ensures the API only accepts valid JSON requests.
- **Model Loading:** The model and tokenizer are loaded into memory once during the server startup using the @app.on_event("startup") decorator. This avoids reloading the files on every single request, which is crucial for high-performance deployment.
- **Prediction Logic:** The /predict endpoint handles the three stages of text classification:
  1. Vectorize the raw input text using the loaded tokenizer.pkl.
  2. Call the model's predict_proba() method to get confidence scores.
  3. Return a structured JSON response with the final sentiment and the prediction score.

### Step 3: Containerization (Docker)

To ensure the environment is portable and consistent, the application is containerized.

1. **Build the Docker Image:** Run the following command from the directory containing the

Dockerfile, app.py, and the .pkl files.
docker build -t nlp-sentiment-api:latest .

- This command builds an image tagged nlp-sentiment-api with the version latest.

2. **Run the Docker Container:** Start the container, mapping the internal port 8000 to the host machine's port 8000.
   docker run -d --name nlp_predictor -p 8000:8000 nlp-sentiment-api:latest

   - The -d runs the container in detached mode (background).

### Step 4: Testing and Documentation

Once the container is running, the API can be accessed from the host machine:

1. **Access Documentation:** Open your web browser and navigate to:
   - **Interactive Documentation (Swagger UI):** http://localhost:8000/docs
2. **Test the Endpoint:** Use the interactive documentation at /docs to test the /predict endpoint.

## 5. Summary of Tools Used

| Tool | Purpose |
|---|---|
| **FastAPI** | High-performance, modern Python web framework for API development. |
| **Uvicorn** | ASGI server for running FastAPI asynchronously. |
| **Docker** | Containerization platform for packaging the application and its environment. |
| **Pydantic** | Data validation and settings management (used for input/output schemas). |
| **Joblib** | Serialization/deserialization library used to load the trained model (.pkl files) efficiently. |
| **scikit-learn** | Mock model (Logistic Regression) and tokenizer (TfidfVectorizer). |